

注：此文档旨在对前期linux学习的归纳总结，区别于每日上传的linux学习记录，该文档内容含大量个人理解向内容，仅用于个人方便应用记忆

Ch5相关：

使用ls-la 命令查询root身份下的文件后，会得到

```
[dmtsai@study ~]$ su - # 先来切换一下身份看看
Password:
Last login: Tue Jun  2 19:32:31 CST 2015 on tty2
[root@study ~]# ls -al
total 48
dr-xr-x---.  5   root    root    4096 May 29 16:08 .
dr-xr-xr-x. 17   root    root    4096 May  4 17:56 ..
-rw-----.  1   root    root    1816 May  4 17:57 anaconda-ks.cfg
-rw-----.  1   root    root     927 Jun  2 11:27 .bash_history
-rw-r--r--.  1   root    root      18 Dec 29 2013 .bash_logout
-rw-r--r--.  1   root    root     176 Dec 29 2013 .bash_profile
-rw-r--r--.  1   root    root     176 Dec 29 2013 .bashrc
drwxr-xr-x.  3   root    root      17 May  6 00:14 .config                               &lt;=范例说明处
drwx-----.  3   root    root      24 May  4 17:59 .dbus
-rw-r--r--.  1   root    root   1864 May  4 18:01 initial-setup-ks.cfg          &lt;=范例说明处
[  1  ][  2  ][  3  ][  4  ][  5  ][  6  ][  7  ]
[ 权限  ][链接][拥有者][群组][文件大小][ 修改日期 ][ 文件名 ]
```

这样的东西

因是自己看，此处只赘述自己需要着重记忆的地方，倘若后续偶然分享，还请观者见谅

档案类型：

【d】是目录

【-】是文件

【l】是链接文件

【b】标识是可供存储的设备

【c】表示为设备文件里面的设备

-rwx-： r表示可读，w表示可写，x表示可执行

三组标识对应：文件拥有者可具备的权限，加入此群组之账号的权限，非本人且没有加入本群组之其他账号的权限

上面提到了文件类型，那么修改文件类型与权限如下

Chgrp：改变文件所属组群 change group

Chown：改变文件所有者 change owner

Chmod：改变文件的权限

在使用chgrp改变所属组群时：

Chgrp 【-R】表示连同修改的目录以下的所有目录都修改为该群组

chgrp 【-R】dirname/filename ...

示例：chgrp users initial-setup-ks-cfg

在使用chown改变文件所有者时：

同上【-R】可以修改目录及以下所有文件的所有者

语法：chown 【-R】 账号名称 文件或目录

Chown 【-R】账号名称：群组名称 文件或目录：旨在同时修改文件所有者与组群

示例：将initial-setup-ks-cfg的所有者改为bin

Chown 【-R】bin intiial-setup-ks.cfg

应用举例：

cp .bashrc .bashrc_test :复制文件

chown 【-R】bin: bin bashrc_text

在使用：chmod修改文件权限时

语法：chmod 【-R】 xyz 文件或目录

实例：chmod 777 .bashrc

语法：chmod u=rwx, g=rwx, o=rwx 文件或目录

语法：chmod a+w 文件或目录 直接所有权限上加w

语法：chmod a-x 文件或目录 如上对应

Ch6对应

绝对路径：由根目录写起，eg. /user/share/doc

相对目录：不由/写起，而是例如由/user/share/doc到/user/share/man下面时，可以写成cd../man 意为相对于目前工作目录的路径

特殊目录：

. 代表此层目录

.. 代表上一层目录

“—” 代表前一个目录

~ 代表 “目前使用者身份” 所在的主文件夹

~x 代表 “X” 这个试用者的主文件夹

常见处理目录的指令：

Cd 变换目录 change Directory

Pwd 显示目前的目录

Mkdir 创建一个新的目录

Rmdir 删除一个空的目录

```
[dmtsai@study ~]$ su - # 先切换身份成为 root 看看！
[root@study ~]# cd [相对路径或绝对路径]
# 最重要的就是目录的绝对路径与相对路径，还有一些特殊目录的符号啰！
[root@study ~]# cd ~dmtsai
# 代表去到 dmtsai 这个使用者的主文件夹，亦即 /home/dmtsai
[root@study dmtsai]# cd ~
# 表示回到自己的主文件夹，亦即是 /root 这个目录
[root@study ~]# cd
# 没有加上任何路径，也还是代表回到自己主文件夹的意思喔！
[root@study ~]# cd ..
# 表示去到目前的上层目录，亦即是 /root 的上层目录的意思；
[root@study /]# cd -
# 表示回到刚刚的那个目录，也就是 /root 啰~
[root@study ~]# cd /var/spool/mail
# 这个就是绝对路径的写法！直接指定要去的完整路径名称！
[root@study mail]# cd ../postfix
# 这个是相对路径的写法，我们由/var/spool/mail 去到/var/spool/postfix 就这样写！
```

关于可执行文件路径的变量：\$PATH

Mv :移动

语法：mv /bin/is /root

文件与目录管理

语法：

ls -a 文件名或目录名： 全部的文件，连同隐藏的文件

-A 文件名或目录名： 全部的文件，但不包括 ... 两个文件夹

-d 同上： 仅列出目录本身，而不是列出目录内的文件数据

-f 同上： 直接列出结果，而不进行排序

-F 同上： 根据文件，目录等信息，给予附加数据结构

*****为可执行文件 /代表目录 =代表socket文件 | 代表FIFO文件

-h 同上： 将文件夹大小以kb，mb的形式列出来

-i 同上：列出inode号码

-l 同上：长数据串行输出，包含文件的属性与权限等等数据

-n 同上：列出uid与gid而非使用者与群组的名称

-R同上：连同子目录内容一起列出来，等于该目录下的所有文件都会显示出来

-r 同上：将排序结果反向输出，例如：原本文件名由大到小，反向则为由小到大

-s 同上：以文件大小排序

-t 同上：以时间排序

--color=never：不要依据文件特性给予颜色显示

--color=always：显示颜色

--full-time：完整时间模式（年，月，日，时，分）输出

--time={atime, ctime}：输出access时间或改变权限属性时间（ctime），而非内容变更时间（modification time）

复制，删除与移动

复制：cp (copy)

语法：cp **【-adfilprsu】** 来源文件 (source) 目标文件 (destination)

```
[root@study ~]# cp [-adfilprsu] 来源文件 (source) 目标文件 (destination)
[root@study ~]# cp [options] source1 source2 source3 .... directory
```

选项与参数：

- a：相当于 -dr --preserve=all 的意思，至于 dr 请参考下列说明；（常用）
- d：若来源文件为链接文件的属性（link file），则复制链接文件属性而非文件本身；
- f：为强制（force）的意思，若目标文件已经存在且无法打开，则移除后再尝试一次；
- i：若目标文件（destination）已经存在时，在覆盖时会先询问动作的进行（常用）
- l：进行硬式链接（hard link）的链接文件创建，而非复制文件本身；
- p：连同文件的属性（权限、用户、时间）一起复制过去，而非使用默认属性（备份常用）；
- r：递归持续复制，用于目录的复制行为；（常用）
- s：复制成为符号链接文件（symbolic link），亦即“捷径”文件；
- u：destination 比 source 旧才更新 destination，或 destination 不存在的情况下才复制。

--preserve=all：除了 -p 的权限相关参数外，还加入 SELinux 的属性，links, xattr 等也复制了。

最后需要注意的，如果来源文件有两个以上，则最后一个目的文件一定要是“目录”才行！

-a -i -r为常用

范例一：用root身份，将主文件夹下的 .bashrc 复制到 /tmp 下，并更名为 bashrc
[root@study ~]# cp ~/.bashrc /tmp/bashrc
[root@study ~]# cp -i ~/.bashrc /tmp/bashrc
cp: overwrite `/tmp/bashrc'? n <==n不覆盖，y为覆盖
重复作两次动作，由于 /tmp 下面已经存在 bashrc 了，加上 -i 选项后，
则在覆盖前会询问使用者是否确定！可以按下 n 或者 y 来二次确认呢！

范例二：变换目录到/tmp，并将/var/log/wtmp复制到/tmp且观察属性：
[root@study ~]# cd /tmp
[root@study tmp]# cp /var/log/wtmp . <==想要复制到目前的目录，最后的 . 不要忘
[root@study tmp]# ls -l /var/log/wtmp wtmp
-rw-rw-r--. 1 root utmp 28416 Jun 11 18:56 /var/log/wtmp
-rw-r--r--. 1 root root 28416 Jun 11 19:01 wtmp
注意上面的特殊字体，在不加任何选项的情况下，文件的某些属性/权限会改变；
这是个很重要的特性！要注意喔！还有，连文件创建的时间也不一样了！
那如果你想要将文件的所有特性都一起复制过来该怎么办？可以加上 -a 喔！如下所示：

```
[root@study tmp]# cp -a /var/log/wtmp wtmp_2
[root@study tmp]# ls -l /var/log/wtmp wtmp_2
-rw-rw-r--. 1 root utmp 28416 Jun 11 18:56 /var/log/wtmp
-rw-rw-r--. 1 root utmp 28416 Jun 11 18:56 wtmp_2
# 瞧了吧！整个数据特性完全一模一样！真是不赖~这就是 -a 的特性！
```

总之，cp复制由种种的文件属性与权限的特性，所以，在复制时，你必须要了解清楚

- 1.是否要完整的保留来源文件的信息？
- 2.来源文件是否为链接文件
- 3.来源文件是否为特殊文件？FIFO / socket
- 4.来源文件否为目录

移除文件或目录：rm

```
[root@study ~]# rm [-fir] 文件或目录
```

选项与参数：

- f : 就是 force 的意思，忽略不存在的文件，不会出现警告讯息；
- i : 互动模式，在删除前会询问使用者是否动作
- r : 递归删除啊！最常用在目录的删除了！这是非常危险的选项！！

范例一：将刚刚在 cp 的范例中创建的 bashrc 删除掉！

```
[root@study ~]# cd /tmp
```

```
[root@study tmp]# rm -i bashrc
```

```
rm: remove regular file `bashrc'? y
```

如果加上 -i 的选项就会主动询问喔，避免你删除到错误的文件名！

范例二：通过万用字符*的帮忙，将/tmp下面开头为bashrc的文件名通通删除：

```
[root@study tmp]# rm -i bashrc*
```

注意那个星号，代表的是 0 到无穷多个任意字符喔！很好用的东西！

范例三：将 cp 范例中所创建的 /tmp/etc/ 这个目录删除掉！

```
[root@study tmp]# rmdir /tmp/etc
```

```
rmdir: failed to remove `/tmp/etc': Directory not empty <== 删不掉啊！因为这不是空的目录
```

```
[root@study tmp]# rm -r /tmp/etc
```

```
rm: descend into directory `/tmp/etc'? y
```

```
rm: remove regular file `/tmp/etc/fstab'? y
```

```
rm: remove regular empty file `/tmp/etc/crypttab'? ^C <== 按下 [ctrl]+c 中断
```

..... (中间省略)

因为身份是 root，默认已经加入了 -i 的选项，所以你要一直按 y 才会删除！

如果不想要继续按 y，可以按下“ [ctrl]-c ”来结束 rm 的工作。

这是一种保护的动作，如果确定要删除掉此目录而不要询问，可以这样做：

```
[root@study tmp]# \rm -r /tmp/etc
```

在指令前加上反斜线，可以忽略掉 alias 的指定选项喔！至于 alias 我们在bash再谈！

拜托！这个范例很可怕！你不要删错了！删除 /etc 系统是会挂掉的！

范例四：删除一个带有 - 开头的文件

```
[root@study tmp]# touch ./-aaa- <==[touch](../Text/index.html#touch)这个指令可以创建空文
```

```
[root@study tmp]# ls -l
```

```
-rw-r--r--. 1 root root 0 Jun 11 19:22 -aaa- <==文件大小为0，所以是空文件
```

```
[root@study tmp]# rm -aaa-
```

```
rm: invalid option -- 'a' <== 因为 "-" 是选项嘛！所以系统误判了！
```

```
Try 'rm ./-aaa-' to remove the file `.-aaa-'. <== 新的 bash 有给建议的
```

```
Try 'rm --help' for more information.
```

```
[root@study tmp]# rm ./-aaa-
```

移动文件与目录，或更名


```
[root@study ~]# mv [-fiu] source destination
[root@study ~]# mv [options] source1 source2 source3 .... directory
```

选项与参数：

- f : force 强制的意思，如果目标文件已经存在，不会询问而直接覆盖；
- i : 若目标文件 (destination) 已经存在时，就会询问是否覆盖！
- u : 若目标文件已经存在，且 source 比较新，才会更新 (update)

范例一：复制一文件，创建一目录，将文件移动到目录中

```
[root@study ~]# cd /tmp
[root@study tmp]# cp ~/.bashrc bashrc
[root@study tmp]# mkdir mvtest
[root@study tmp]# mv bashrc mvtest
```

将某个文件移动到某个目录去，就是这样做！

范例二：将刚刚的目录名称更名为 mvtest2

```
[root@study tmp]# mv mvtest mvtest2 &lt;== 这样就更名了！简单~
```

其实在 Linux 下面还有个有趣的指令，名称为 rename ，

该指令专职进行多个文件名的同时更名，并非针对单一文件名变更，与mv不同。请man rename。

范例三：再创建两个文件，再全部移动到 /tmp/mvtest2 当中

```
[root@study tmp]# cp ~/.bashrc bashrc1
[root@study tmp]# cp ~/.bashrc bashrc2
[root@study tmp]# mv bashrc1 bashrc2 mvtest2
```

注意到这边，如果有多个来源文件或目录，则最后一个目标文件一定是“目录！”

意思是说，将所有数据移动到该目录的意思！

cp ~/tmp

cp ~/.bashrc bashrc

cp ~/.bashrc bashrc1

cp ~/.bashrc bashrc2

mkdir mvtest

mv mvtest mvtest2

mv bashrc bashrc1 bashrc2 mvtest2

在tmp文件夹下创建了bashrc...三个文件放到了其下用mkdir新创建的空目录mvtest更名为mvtest2下

至此，cp ,rm,mv 三个常用的文件/文件夹 复制，移除，更名移动命令已解决

文件内容查阅

- cat 由第一行开始显示文件内容
- tac 从最后一行开始显示，可以看出 tac 是 cat 的倒着写！
- nl 显示的时候，顺道输出行号！
- more 一页一页的显示文件内容
- less 与 more 类似，但是比 more 更好的是，他可以往前翻页！
- head 只看头几行
- tail 只看尾巴几行
- od 以二进制的方式读取文件内容！

cat语法：

```
[root@study ~]# cat [-AbEnTv]
```

选项与参数：

- A : 相当于 -vET 的整合选项，可列出一些特殊字符而不是空白而已；
- b : 列出行号，仅针对非空白行做行号显示，空白行不标行号！
- E : 将结尾的断行字符 \$ 显示出来；
- n : 打印出行号，连同空白行也会有行号，与 -b 的选项不同；
- T : 将 [tab] 按键以 ^I 显示出来；
- v : 列出一些看不出来的特殊字符

范例一：检阅 /etc/issue 这个文件的内容

```
[root@study ~]# cat /etc/issue
\S
Kernel \r on an \m
```

范例二：承上题，如果还要加印行号呢？

```
[root@study ~]# cat -n /etc/issue
 1  \S
 2  Kernel \r on an \m
 3
```

所以这个文件有三行！看到了吧！可以印出行号呢！这对于大文件要找某个特定的行时，有点用处！

如果不想编排空白行的行号，可以使用“cat -b /etc/issue”，自己测试看看：

范例三：将 /etc/man_db.conf 的内容完整的显示出来（包含特殊字符）

```
[root@study ~]# cat -A /etc/man_db.conf
# $
.... (中间省略) ....
MANPATH_MAP^I/bin^I^I^I/usr/share/man$
MANPATH_MAP^I/usr/bin^I^I^I/usr/share/man$
MANPATH_MAP^I/sbin^I^I^I/usr/share/man$
MANPATH_MAP^I/usr/sbin^I^I^I/usr/share/man$
..... (下面省略) .....
```

上面的结果限于篇幅，鸟哥删除掉很多数据了。另外，输出的结果并不会特殊字体，

鸟哥上面的特殊字体是要让您发现差异点在哪里就是了。基本上，在一般的环境中，

使用 [tab] 与空白键的效果差不多，都是一堆空白啊！我们无法知道两者的差别。

此时使用 cat -A 就能够发现那些空白的地方是啥鬼东西了！[tab]会以 ^I 表示，

断行字符则是以 \$ 表示，所以你可以发现每一行后面都是 \$ 啊！不过断行字符

在Windows/Linux则不太相同，Windows的断行字符是 ^M\$ 啰。

这部分我们会在[第九章 vim 软件](../Text/index.html#tips_dos)的介绍时，再次的说明到喔！

tac为反向展开，形同cat

- nl (添加行号打印)

```
[root@study ~]# nl [-bnw] 文件
选项与参数：
-b  : 指定行号指定的方式，主要有两种：
      -b a  : 表示不论是否为空行，也同样列出行号（类似 cat -n）；
      -b t  : 如果有空行，空的那一行不要列出行号（默认值）；
-n  : 列出行号表示的方法，主要有三种：
      -n ln  : 行号在屏幕的最左方显示；
      -n rn  : 行号在自己字段的最右方显示，且不加 0 ；
      -n rz  : 行号在自己字段的最右方显示，且加 0 ；
-w  : 行号字段的占用的字符数。

范例一：用 nl 列出 /etc/issue 的内容
[root@study ~]# nl /etc/issue
 1  \S
 2  Kernel \r on an \m

# 注意看，这个文件其实有三行，第三行为空白（没有任何字符），
# 因为他是空白行，所以 nl 不会加上行号喔！如果确定要加上行号，可以这样做：

[root@study ~]# nl -b a /etc/issue
 1  \S
 2  Kernel \r on an \m
 3

# 呵呵！行号加上来啰～那么如果要让行号前面自动补上 0 呢？可这样

[root@study ~]# nl -b a -n rz /etc/issue
000001  \S
000002  Kernel \r on an \m
000003
# 嘿嘿！自动在自己字段的地方补上 0 了～默认字段是六位数，如果想要改成 3 位数？

[root@study ~]# nl -b a -n rz -w 3 /etc/issue
001  \S
002  Kernel \r on an \m
003
# 变成仅有 3 位数啰～
```

nl 可以将输出的文件内容自动的加上行号！其默认的结果与 cat -n 有点不太一样，nl 可以将行号做比较多的显示设计，包括位数与是否自动补齐 0 等的功能呢。

nl 为行号显示，目的旨在更多样式的显示行数

上述三种检视均为一口气全部显示，会导致过长的文件时无法看清前面的数据

可翻页显示：

more (一页一页显示)

语法：more 文件

运行时，可以通过

空格键 下翻一页

enter 下翻一行

/字串 搜寻字串

: f 立刻显示出文件名与目前显示的行数

q 代表立刻离开move

b 往回翻页

less 一页一页翻动，且可上下翻动

语法同上

- 空白键：向下翻动一页；

6.3 文件内容查阅

鸟哥的 Linux 私房菜：基础学习篇 第四版

- [pagedown]：向下翻动一页；
- [pageup]：向上翻动一页；
- /字串：向下搜寻“字串”的功能；
- ?字串：向上搜寻“字串”的功能；
- n：重复前一个搜寻（与/或?有关！）
- N：反向的重复前一个搜寻（与/或?有关！）
- g：前进到这个数据的第一行去；
- G：前进到这个数据的最后一行去（注意大小写）；
- q：离开 less 这个程序；

数据提取

head 语法：head -n number 文件

tail 语法同上

od 显示非纯文本数据

```
[root@study ~]# od [-t TYPE] 文件
```

选项或参数：

-t : 后面可以接各种“类型 (TYPE)”的输出，例如：

a : 利用默认的字符来输出；

c : 使用 ASCII 字符来输出

d[size] : 利用十进制 (decimal) 来输出数据，每个整数占用 size Bytes ；

f[size] : 利用浮点数值 (floating) 来输出数据，每个数占用 size Bytes ；

o[size] : 利用八进制 (octal) 来输出数据，每个整数占用 size Bytes ；

x[size] : 利用十六进制 (hexadecimal) 来输出数据，每个整数占用 size Bytes ；

范例一：请将/usr/bin/passwd的内容使用ASCII方式来展现！

```
[root@study ~]# od -t c /usr/bin/passwd
```

```
00000000 177 E L F 002 001 001 \0 \0 \0 \0 \0 \0 \0 \0
00000020 003 \0 > \0 001 \0 \0 \0 364 3 \0 \0 \0 \0 \0 \0
00000040 @ \0 \0 \0 \0 \0 \0 \0 x e \0 \0 \0 \0 \0 \0
00000060 \0 \0 \0 \0 @ \0 8 \0 \t \0 @ \0 035 \0 034 \0
00000100 006 \0 \0 \0 005 \0 \0 \0 @ \0 \0 \0 \0 \0 \0
..... (后面省略) .....
```

最左边第一栏是以 8 进位来表示Bytes数。以上面范例来说，第二栏00000020代表开头是
第 16 个 bytes (2x8) 的内容之意。

范例二：请将/etc/issue这个文件的内容以8进位列出储存值与ASCII的对照表

```
[root@study ~]# od -t oCc /etc/issue
```

```
00000000 134 123 012 113 145 162 156 145 154 040 134 162 040 157 156 040
          \ S \n K e r n e l \ r o n
00000020 141 156 040 134 155 012 012
          a n \ m \n \n
00000027
```

如上所示，可以发现每个字符可以对应到的数值为何！要注意的是，该数值是 8 进位喔！

例如 S 对应的记录数值为 123，转成十进制：1x8^2+2x8+3=83。

修改文件时间或创建新文件 touch

首先介绍ls查询中的时间参数

mtime (modification time) :当该文件的内容数据变更时，就会更新该时间

ctime (status time) :当该文件的状态更新时，就会更新该时间

atime (access time) : 当该文件的内容被提取时，就会更新该时间

ls默认显示文件的mtime

查询时可用分号隔开查询内容

date; ls -l /etc/文件名; ls -l --time=atime /文件名;

```
[root@study ~]# date; ls -l /etc/man_db.conf ; ls -l --time=atime /etc/man_db.conf ; \
```

```
&gt; ls -l --time=ctime /etc/man_db.conf # 这两行其实是同一行喔！用分号隔开
```

```
Tue Jun 16 00:43:17 CST 2015 # 目前的时间啊！
```

```
-rw-r--r--. 1 root root 5171 Jun 10 2014 /etc/man_db.conf # 在 2014/06/10 创建的内容 (mti
```

```
-rw-r--r--. 1 root root 5171 Jun 15 23:46 /etc/man_db.conf # 在 2015/06/15 读取过内容 (ati
```

```
-rw-r--r--. 1 root root 5171 May 4 17:54 /etc/man_db.conf # 在 2015/05/04 更新过状态 (cti
```

```
# 为了要让数据输出比较好看，所以鸟哥将三个指令同时依序执行，三个指令中间用分号 (;) 隔开即可。
```

touch 语法：

```
[root@study ~]# touch [-acdm] 文件
选项与参数：
-a   : 仅修订 access time；
-c   : 仅修改文件的时间，若该文件不存在则不创建新文件；
-d   : 后面可以接欲修订的日期而不用目前的日期，也可以使用 --date="日期或时间"
-m   : 仅修改 mtime；
-t   : 后面可以接欲修订的时间而不用目前的时间，格式为[YYYYMMDDhhmm]

范例一：新建一个空的文件并观察时间
[dmtsai@study ~]# cd /tmp
[dmtsai@study tmp]# touch testtouch
[dmtsai@study tmp]# ls -l testtouch
-rw-rw-r--. 1 dmtsai dmtsai 0 Jun 16 00:45 testtouch
# 注意到，这个文件的大小是 0 呢！在默认的状态下，如果 touch 后面有接文件，
# 则该文件的三个时间（atime/ctime/mtime）都会更新为目前的时间。若该文件不存在，
# 则会主动的创建一个新的空的文件喔！例如上面这个例子！

范例二：将 ~/.bashrc 复制成为 bashrc，假设复制完全的属性，检查其日期
[dmtsai@study tmp]# cp -a ~/.bashrc bashrc
[dmtsai@study tmp]# date; ll bashrc; ll --time=atime bashrc; ll --time=ctime bashrc
Tue Jun 16 00:49:24 CST 2015                                &lt;==这是目前的时间
-rw-r--r--. 1 dmtsai dmtsai 231 Mar  6 06:06 bashrc        &lt;==这是 mtime
-rw-r--r--. 1 dmtsai dmtsai 231 Jun 15 23:44 bashrc        &lt;==这是 atime
-rw-r--r--. 1 dmtsai dmtsai 231 Jun 16 00:47 bashrc        &lt;==这是 ctime
```

touch最被常使用的情况是：

创建一个空的文件

将某个文件日期修改为目前

```
[root@study ~]# touch [-acdm] 文件
选项与参数：
-a   : 仅修订 access time；
-c   : 仅修改文件的时间，若该文件不存在则不创建新文件；
-d   : 后面可以接欲修订的日期而不用目前的日期，也可以使用 --date="日期或时间"
-m   : 仅修改 mtime；
-t   : 后面可以接欲修订的时间而不用目前的时间，格式为[YYYYMMDDhhmm]

范例一：新建一个空的文件并观察时间
[dmtsai@study ~]# cd /tmp
[dmtsai@study tmp]# touch testtouch
[dmtsai@study tmp]# ls -l testtouch
-rw-rw-r--. 1 dmtsai dmtsai 0 Jun 16 00:45 testtouch
# 注意到，这个文件的大小是 0 呢！在默认的状态下，如果 touch 后面有接文件，
# 则该文件的三个时间（atime/ctime/mtime）都会更新为目前的时间。若该文件不存在，
# 则会主动的创建一个新的空的文件喔！例如上面这个例子！

范例二：将 ~/.bashrc 复制成为 bashrc，假设复制完全的属性，检查其日期
[dmtsai@study tmp]# cp -a ~/.bashrc bashrc
[dmtsai@study tmp]# date; ll bashrc; ll --time=atime bashrc; ll --time=ctime bashrc
Tue Jun 16 00:49:24 CST 2015                                &lt;==这是目前的时间
-rw-r--r--. 1 dmtsai dmtsai 231 Mar  6 06:06 bashrc        &lt;==这是 mtime
-rw-r--r--. 1 dmtsai dmtsai 231 Jun 15 23:44 bashrc        &lt;==这是 atime
-rw-r--r--. 1 dmtsai dmtsai 231 Jun 16 00:47 bashrc        &lt;==这是 ctime
```

观察文件类型 file

语法：file 文件

指令/文件的搜寻

whereis

语法:

```
[root@study ~]# whereis [-bmsu] 文件或目录名
选项与参数:
-l      :可以列出 whereis 会去查询的几个主要目录而已
-b      :只找 binary 格式的文件
-m      :只找在说明文档 manual 路径下的文件
-s      :只找 source 来源文件
-u      :搜寻不在上述三个项目当中的其他特殊文件

范例一:请找出 ifconfig 这个文件名
[root@study ~]# whereis ifconfig
ifconfig: /sbin/ifconfig /usr/share/man/man8/ifconfig.8.gz

范例二:只找出跟 passwd 有关的“说明文档”文件名 (man page)
[root@study ~]# whereis passwd      # 全部的文件名通通列出来!
passwd: /usr/bin/passwd /etc/passwd /usr/share/man/man1/passwd.1.gz /usr/share/man/man5/p
[root@study ~]# whereis -m passwd    # 只有在 man 里面的文件名才抓出来!
passwd: /usr/share/man/man1/passwd.1.gz /usr/share/man/man5/passwd.5.gz
```

locate 查询关键词说是

```
[root@study ~]# locate [-ir] keyword
选项与参数:
-i      :忽略大小写的差异;
-c      :不输出文件名,仅计算找到的文件数量
-l      :仅输出几行的意思,例如输出五行则是 -l 5
-S      :输出 locate 所使用的数据库文件的相关信息,包括该数据库纪录的文件/目录数量等
-r      :后面可接正则表达式的显示方式

范例一:找出系统中所有与 passwd 相关的文件名,且只列出 5 个
[root@study ~]# locate -l 5 passwd
/etc/passwd
/etc/passwd-
/etc/pam.d/passwd
/etc/security/opasswd
/usr/bin/gpasswd

范例二:列出 locate 查询所使用的数据库文件之文件名与各数据数量
[root@study ~]# locate -S
Database /var/lib/mlocate/mlocate.db:
      8,086 directories      # 总纪录目录数
    109,605 files            # 总纪录文件数
    5,190,295 Bytes in file names
    2,349,150 Bytes used to store database
```

关键词搜寻很好用,但是其只搜索当日更新时的数据,手动更新如下

输入 updatedb 就好

find

语法：

```
[root@study ~]# find [PATH] [option] [action]
```

选项与参数：

- 1\、与时间有关的选项：共有 `-atime`、`-ctime` 与 `-mtime`，以 `-mtime` 说明
- `-mtime n`：n 为数字，意义为在 n 天之前的“一天之内”被更动过内容的文件；
 - `-mtime +n`：列出在 n 天之前（不含 n 天本身）被更动过内容的文件文件名；
 - `-mtime -n`：列出在 n 天之内（含 n 天本身）被更动过内容的文件文件名。
 - `-newer file`：file 为一个存在的文件，列出比 file 还要新的文件文件名

范例一：将过去系统上面 24 小时内有变动过内容（mtime）的文件列出

```
[root@study ~]# find / -mtime 0
```

- # 那个 0 是重点！0 代表目前的时间，所以，从现在开始到 24 小时前，
- # 有变动过内容的文件都会被列出来！那如果是三天前的 24 小时内？
- # `find / -mtime 3` 有变动过的文件都被列出的意思！

范例二：寻找 /etc 下面的文件，如果文件日期比 /etc/passwd 新就列出

```
[root@study ~]# find /etc -newer /etc/passwd
```

- # `-newer` 用在分辨两个文件之间的新旧关系是很有用的！

选项与参数：

2\、与使用者或群组名称有关的参数：

- `-uid n`：n 为数字，这个数字是使用者的帐号 ID，亦即 UID，这个 UID 是记录在 /etc/passwd 里面与帐号名称对应的数字。这方面我们会在第四篇介绍。
- `-gid n`：n 为数字，这个数字是群组名称的 ID，亦即 GID，这个 GID 记录在 /etc/group，相关的介绍我们会第四篇说明～
- `-user name`：name 为使用者帐号名称喔！例如 dmtsai
- `-group name`：name 为群组名称喔，例如 users；
- `-nouser`：寻找文件的拥有者不存在 /etc/passwd 的人！
- `-nogroup`：寻找文件的拥有群组不存在于 /etc/group 的文件！
当你自行安装软件时，很可能该软件的属性当中并没有文件拥有者，这是可能的！在这个时候，就可以使用 `-nouser` 与 `-nogroup` 搜寻。

范例三：搜寻 /home 下面属于 dmtsai 的文件

```
[root@study ~]# find /home -user dmtsai
```

- # 这个东西也很有用的～当我们要找出任何一个使用者在系统当中的所有文件时，
- # 就可以利用这个指令将属于某个使用者的所有文件都找出来喔！

范例四：搜寻系统中不属于任何人的文件

```
[root@study ~]# find / -nouser
```

- # 通过这个指令，可以轻易的就找出那些不太正常的文件。如果有找到不属于系统任何人的文件时，
- # 不要太紧张，那有时候是正常的～尤其是你曾经以源代码自行编译软件时。

ch6重要复习

6.6 极重要的复习！权限与指令间的关系

我们知道权限对于使用者帐号来说是非常重要的，因为他可以限制使用者能不能读取/创建/删除/修改文件或目录！在这一章我们介绍了很多文件系统的管理指令，第五章则介绍了很多文件权限的意义。在这个小节当中，我们就将这两者结合起来，说明一下什么指令在什么样的权限下才能够运行吧！^_^

一、让使用者能进入某目录成为“可工作目录”的基本权限为何？

- 可使用的指令：例如 `cd` 等变换工作目录的指令；
- 目录所需权限：使用者对这个目录至少需要具有 `x` 的权限
- 额外需求：如果使用者想要在这个目录内利用 `ls` 查阅文件名，则使用者对此目录还需要 `r` 的权限。

二、使用者在某个目录内读取一个文件的基本权限为何？

- 可使用的指令：例如本章谈到的 `cat`, `more`, `less` 等等
- 目录所需权限：使用者对这个目录至少需要具有 `x` 权限；
- 文件所需权限：使用者对文件至少需要具有 `r` 的权限才行！

三、让使用者可以修改一个文件的基本权限为何？

- 可使用的指令：例如 `nano` 或未来要介绍的 `vi` 编辑器等；
- 目录所需权限：使用者在该文件所在的目录至少要有 `x` 权限；
- 文件所需权限：使用者对该文件至少要有 `r, w` 权限

四、让一个使用者可以创建一个文件的基本权限为何？

- 目录所需权限：使用者在该目录要具有 `w, x` 的权限，重点在 `w` 啦！

五、让使用者进入某目录并执行该目录下的某个指令之基本权限为何？

- 目录所需权限：使用者在该目录至少要有 `x` 的权限；
- 文件所需权限：使用者在该文件至少需要具有 `x` 的权限

- 绝对路径：“一定由根目录 / 写起”；相对路径：“不由 / 写起，而是由相对当前目录写起”
- 特殊目录有：., .., -, ~, ~account 需要注意；
- 与目录相关的指令有：cd, mkdir, rmdir, pwd 等重要指令；
- rmdir 仅能删除空目录，要删除非空目录需使用“rm -r”指令；
- 使用者能使用的指令是依据 PATH 变量所规定的目录去搜寻的；
- ls 可以检视文件的属性，尤其 -d, -a, -l 等选项特别重要！
- 文件的复制、删除、移动可以分别使用：cp, rm, mv 等指令来操作；
- 检查文件的内容（读档）可使用的指令包括有：cat, tac, nl, more, less, head, tail, od 等
- cat -n 与 nl 均可显示行号，但默认的情况下，空白行会不会编号并不相同；
- touch 的目的在修改文件的时间参数，但亦可用来创建空文件；
- 一个文件记录的时间参数有三种，分别是 access time (atime), status time (ctime), modification time (mtime)，ls 默认显示的是 mtime。
- 除了传统的 rwx 权限之外，在 Ext2/Ext3/Ext4/xfs 文件系统中，还可以使用 chattr 与 lsattr 设置及观察隐藏属性。常见的包括只能新增数据的 +a 与完全不能更动文件的 +i 属性。
- 新建文件/目录时，新文件的默认权限使用 umask 来规范。默认目录完全权限为 drwxrwxrwx，文件则为 -rw-rw-rw-。
- 文件具有 SUID 的特殊权限时，代表当使用者执行此一 binary 程序时，在执行过程中使用者会暂时具有程序拥有者的权限
- 目录具有 SGID 的特殊权限时，代表使用者在这个目录下面新建的文件之群组都会与该目录的群组名称相同。
- 目录具有 SBIT 的特殊权限时，代表在该目录下使用者创建的文件只有自己与 root 能够删除！
- 观察文件的类型可以使用 file 指令来观察；
- 搜寻指令的完整文件名可用 which 或 type，这两个指令都是通过 PATH 变量来搜寻文件名；
- 搜寻文件的完整文件名可以使用 whereis 找特定目录或 locate 到数据库去搜寻，而不实际搜寻文件系统；
- 利用 find 可以加入许多选项来直接查询文件系统，以获得自己想要知道的文件名。

Ch8 压缩/解压缩/打包相关

linux 常用压缩类型：

```
*.Z      compress 程序压缩的文件；
*.zip    zip 程序压缩的文件；
*.gz     gzip 程序压缩的文件；
*.bz2    bzip2 程序压缩的文件；
*.xz     xz 程序压缩的文件；
*.tar     tar 程序打包的数据，并没有压缩过；
*.tar.gz  tar 程序打包的文件，其中并且经过 gzip 的压缩
*.tar.bz2 tar 程序打包的文件，其中并且经过 bzip2 的压缩
*.tar.xz  tar 程序打包的文件，其中并且经过 xz 的压缩
```

gzip:

```
[dmtsai@study ~]$ gzip [-cdtv#] 文件名
[dmtsai@study ~]$ zcat 文件名.gz
选项与参数：
-c   : 将压缩的数据输出到屏幕上，可通过数据流重向来处理；
-d   : 解压缩的参数；
-t   : 可以用来检验一个压缩文件的一致性~看看文件有无错误；
-v   : 可以显示出原文件/压缩文件的压缩比等信息；
-#   : # 为数字的意思，代表压缩等级，-1 最快，但是压缩比最差、-9 最慢，但是压缩比最好！默认是 -6
```

范例一：找出 /etc 下面（不含子目录）容量最大的文件，并将它复制到 /tmp，然后以 gzip 压缩

```
[dmtsai@study ~]$ ls -ldSr /etc/* # 忘记选项意义？请自行 man 啰！
..... (前面省略) .....
-rw-r--r--. 1 root root 25213 Jun 10 2014 /etc/dnsmasq.conf
-rw-r--r--. 1 root root 69768 May 4 17:55 /etc/ld.so.cache
-rw-r--r--. 1 root root 670293 Jun 7 2013 /etc/services

[dmtsai@study ~]$ cd /tmp
[dmtsai@study tmp]$ cp /etc/services .
[dmtsai@study tmp]$ gzip -v services
services: 79.7% -- replaced with services.gz
[dmtsai@study tmp]$ ll /etc/services /tmp/services*
-rw-r--r--. 1 root root 670293 Jun 7 2013 /etc/services
-rw-r--r--. 1 dmtsai dmtsai 136088 Jun 30 18:40 /tmp/services.gz
```

zcat/zmore/zless 指使用cat/more/less来读取压缩数据输出在屏幕上

复习：cat为全部显示，more为可向下翻页，less为可上下翻页

解压缩gzip: gzip -d 文件名

```
范例二：由于 services 是文本文件，请将范例一的压缩文件的内容读出来！
[dmtsai@study tmp]$ zcat services.gz
# 由于 services 这个原本的文件是文本文件，因此我们可以尝试使用 zcat/zmore/zless 去读取！
# 此时屏幕上会显示 servcies.gz 解压缩之后的原始文件内容！
```

```
范例三：将范例一的文件解压缩
[dmtsai@study tmp]$ gzip -d services.gz
# 鸟哥不要使用 gunzip 这个指令，不好背！使用 gzip -d 来进行解压缩！
# 与 gzip 相反，gzip -d 会将原本的 .gz 删除，回复到原本的 services 文件。
```

```
范例四：将范例三解开的 services 用最佳的压缩比压缩，并保留原本的文件
[dmtsai@study tmp]$ gzip -9 -c services &gt; services.gz
```

```
范例五：由范例四再次创建的 services.gz 中，找出 http 这个关键字在哪几行？
[dmtsai@study tmp]$ zgrep -n 'http' services.gz
14:# http://www.iana.org/assignments/port-numbers
89:http 80/tcp www www-http # WorldWideWeb HTTP
90:http 80/udp www www-http # HyperText Transfer Protocol
..... (下面省略) .....
```

bzip2 取代gzip 提供更大的压缩比 语法同上

xz 是bzip2的进阶，更大压缩比

打包指令：tar 将多个文件或目录包成一个大文件

```
[dmtsai@study ~]$ tar [-z#124;-j#124;-J] [cv] [-f 待创建的新文件名] filename... &lt;==打包
[dmtsai@study ~]$ tar [-z#124;-j#124;-J] [tv] [-f 既有的 tar文件名] &lt;==?
[dmtsai@study ~]$ tar [-z#124;-j#124;-J] [xv] [-f 既有的 tar文件名] [-C 目录] &lt;==解
选项与参数：
-c    : 创建打包文件，可搭配 -v 来察看过程中被打包的文件名 (filename)
-t    : 察看打包文件的内容含有哪些文件名，重点在察看“文件名”就是了；
-x    : 解打包或解压缩的功能，可以搭配 -C (大写) 在特定目录解开
      特别留意的是， -c, -t, -x 不可同时出现在一串命令行中。
-z    : 通过 gzip 的支持进行压缩/解压缩：此时文件名最好为 *.tar.gz
-j    : 通过 bzip2 的支持进行压缩/解压缩：此时文件名最好为 *.tar.bz2
-J    : 通过 xz 的支持进行压缩/解压缩：此时文件名最好为 *.tar.xz
      特别留意， -z, -j, -J 不可以同时出现在一串命令行中
-v    : 在压缩/解压缩的过程中，将正在处理的文件名显示出来！
-f filename: -f 后面要立刻接要被处理的文件名！建议 -f 单独写一个选项啰！（比较不会忘记）
-C 目录    : 这个选项用在解压缩，若要在特定目录解压缩，可以使用这个选项。
```

其他后续练习会使用到的选项介绍：

```
-p (小写) : 保留备份数据的原本权限与属性，常用于备份 (-c) 重要的配置文件
-P (大写) : 保留绝对路径，亦即允许备份数据中含有根目录存在之意；
--exclude=FILE: 在压缩的过程中，不要将 FILE 打包！
```

语法：

tar 压缩：tar -jcv -f (filename 文件名)

tar 查询：tar -jtv -f (filename 文件名)

tar 解压缩：tar -jxv -f (filename 欲解压缩的目录)

压缩：


```
[dmtsai@study ~]$ su - # 因为备份 /etc 需要 root 的权限，否则会出现一堆错误
[root@study ~]# time tar -zpcv -f /root/etc.tar.gz /etc
tar: Removing leading '/' from member names &lt;==注意这个警告讯息
/etc/
.... (中间省略) ....
/etc/hostname
/etc/aliases.db

real    0m0.799s   # 多了 time 会显示程序运行的时间！看 real 就好了！花去了 0.799s
user    0m0.767s
sys     0m0.046s
# 由于加上 -v 这个选项，因此正在作用中的文件名就会显示在屏幕上。
# 如果你可以翻到第一页，会发现出现上面的错误讯息！下面会讲解。
# 至于 -p 的选项，重点在于“保留原本文件的权限与属性”之意。

[root@study ~]# time tar -jpcv -f /root/etc.tar.bz2 /etc
.... (前面省略) ....
real    0m1.913s
user    0m1.881s
sys     0m0.038s
[root@study ~]# time tar -Jpcv -f /root/etc.tar.xz /etc
.... (前面省略) ....
real    0m9.023s
user    0m8.984s
sys     0m0.086s
# 显示的讯息会跟上面一模一样啰！不过时间会花比较多！使用了 -J 时，会花更多时间

[root@study ~]# ll /root/etc*
-rw-r--r--. 1 root root 6721809 Jul  1 00:16 /root/etc.tar.bz2
-rw-r--r--. 1 root root 7758826 Jul  1 00:14 /root/etc.tar.gz
-rw-r--r--. 1 root root 5511500 Jul  1 00:16 /root/etc.tar.xz
[root@study ~]# du -sm /etc
28      /etc  # 实际目录约占有 28MB 的意思！
```

查询：

```
[root@study ~]# tar -jtv -f /root/etc.tar.bz2
.... (前面省略) ....
-rw-r--r-- root/root      131 2015-05-25 17:48 etc/locale.conf
-rw-r--r-- root/root       19 2015-05-04 17:56 etc/hostname
-rw-r--r-- root/root    12288 2015-05-04 17:59 etc/aliases.db
```

解压缩：

```
[root@study ~]# tar -jxv -f /root/etc.tar.bz2
[root@study ~]# ll
.... (前面省略) ....
drwxr-xr-x. 131 root root   8192 Jun 26 22:14 etc
.... (后面省略) ....
```

解开文件的其中一个文件

```
# 1\, 先找到我们要的文件名, 假设解开 shadow 文件好了:
[root@study ~]# tar -jtv -f /root/etc.tar.bz2 &#124; grep 'shadow'
----- root/root          721 2015-06-17 00:20 etc/gshadow
----- root/root          1183 2015-06-17 00:20 etc/shadow-
----- root/root          1210 2015-06-17 00:20 etc/shadow  &lt;==这是我们要的!
----- root/root          707 2015-06-17 00:20 etc/gshadow-
# 先搜寻重要的文件名! 其中那个 grep 是“撷取”关键字的功能! 我们会在第三篇说明!
# 这里您先有个概念即可! 那个管线 &#124; 配合 grep 可以撷取关键字的意思!

# 2\, 将该文件解开! 语法与实际作法如下:
[root@study ~]# tar -jxv -f 打包档.tar.bz2 待解开文件名
[root@study ~]# tar -jxv -f /root/etc.tar.bz2 etc/shadow
etc/shadow
[root@study ~]# ll etc
total 4
-----. 1 root root 1210 Jun 17 00:20 shadow
# 很有趣! 此时只会解开一个文件而已! 不过, 重点是那个文件名! 你要找到正确的文件名。
# 在本例中, 你不能写成 /etc/shadow ! 因为记录在 etc.tar.bz2 内的并没有 / 之故!
```

tarfile,tarball

tarfile为仅打包而已

tarball为还有进行压缩的支持

Ch9vi相关与vim相关

^ _ ^

一般指令模式/编辑模式/命令行模式

简单的说, 我们可以将这三个模式想成下面的图示来表示:

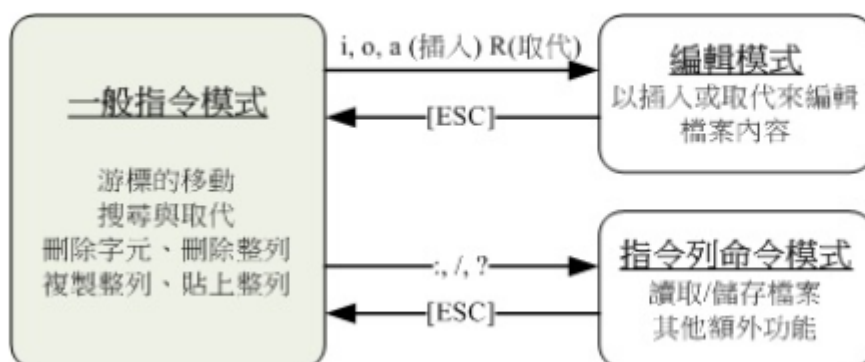


图9.2.1、vi三种模式的相互关系

基本上 vi 共分为三种模式，分别是“一般指令模式”、“编辑模式”与“命令行命令模式”。这三种模式的作用分别是：

- 一般指令模式（command mode）

以 vi 打开一个文件就直接进入一般指令模式了（这是默认的模式，也简称为一般模式）。在这个模式中，你可以使用“上下左右”按键来移动光标，你可以使用“删除字符”或“删除整列”来处理文件内容，也可以使用“复制、贴上”来处理你的文件数据。

- 编辑模式（insert mode）

在一般指令模式中可以进行删除、复制、贴上等等的动作，但是却无法编辑文件内容的！要等到你按下“i, l, o, O, a, A, r, R”等任何一个字母之后才会进入编辑模式。注意了！通常在 Linux 中，按下这些按键时，在画面的左下方会出现“INSERT 或 REPLACE”的字样，此时才可以进行编辑。而如果要回到一般指令模式时，则必须要按下“Esc”这个按键即可退出编辑模式。

- 命令行命令模式（command-line mode）

在一般模式当中，输入“:/?”三个中的任何一个按钮，就可以将光标移动到最下面那一列。在这个模式当中，可以提供你“搜寻数据”的动作，而读取、存盘、大量取代字符、离开 vi、显示行号等等的动作则是在此模式中达成的！

vi语法：

/bin/vi 欲创建文件名

按下i进入编辑模式，开始编辑

: wq存盘

ls -l 查看

p461-p464 由vi/vim的常用命令详解

ch10 认识与学习bash

- [Tab] 接在一串指令的第一个字的后面，则为命令补全；
- [Tab] 接在一串指令的第二个字以后时，则为“文件补齐”！
- 若安装 **bash-completion** 软件，则在某些指令后面使用 [tab] 按键时，可以进行“选项/参数的补齐”功能！

抱歉ch10的东西属实太多且并非当前的必须内容，故暂且跳过

ch11 正则表达式与文件格式化处理

基础正则表达式

摘取命令cut/grep:

cut用于在整齐格式时取出想要的文段

```
[dmtsai@study ~]$ cut -d'分隔字符' -f fields &lt;==用于有特定分隔字符
[dmtsai@study ~]$ cut -c 字符区间 &lt;==用于排列整齐的消息
选项与参数:
-d : 后面接分隔字符。与 -f 一起使用;
-f : 依据 -d 的分隔字符将一段消息分区成为数段,用 -f 取出第几段的意思;
-c : 以字符 (characters) 的单位取出固定字符区间;

范例一: 将 PATH 变量取出,我要找出第五个路径。
[dmtsai@study ~]$ echo ${PATH}
/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/dmtsai/.local/bin:/home/dmtsai/b
#      1      &#124;      2      &#124;      3      &#124;      4      &#124;      5

[dmtsai@study ~]$ echo ${PATH} &#124; cut -d ':' -f 5
# 如同上面的数字显示,我们是以 ":" 作为分隔,因此会出现 /home/dmtsai/.local/bin
# 那么如果想要列出第 3 与第 5 呢?,就是这样:
[dmtsai@study ~]$ echo ${PATH} &#124; cut -d ':' -f 3,5

范例二: 将 export 输出的消息,取得第 12 字符以后的所有字串
[dmtsai@study ~]$ export
declare -x HISTCONTROL="ignoredups"
declare -x HISTSIZE="1000"
declare -x HOME="/home/dmtsai"
declare -x HOSTNAME="study.centos.vbird"
..... (其他省略) .....
# 注意看,每个数据都是排列整齐的输出!如果我们不想要 " declare -x "时,就得这么做:

[dmtsai@study ~]$ export &#124; cut -c 12-
HISTCONTROL="ignoredups"
HISTSIZE="1000"
HOME="/home/dmtsai"
HOSTNAME="study.centos.vbird"
..... (其他省略) .....
# 知道怎么回事了吧?用 -c 可以处理比较具有格式的输出数据!
# 我们还可以指定某个范围的值,例如第 12-20 的字符,就是 cut -c 12-20 等等!

范例三: 用 last 将显示的登陆者的信息中,仅留下使用者大名
[dmtsai@study ~]$ last
root pts/1 192.168.201.101 Sat Feb 7 12:35 still logged in
root pts/1 192.168.201.101 Fri Feb 6 12:13 - 18:46 (06:33)
root pts/1 192.168.201.254 Thu Feb 5 22:37 - 23:53 (01:16)
# last 可以输出"帐号/终端机/来源/日期时间"的数据,并且是排列整齐的

[dmtsai@study ~]$ last &#124; cut -d ' ' -f 1
# 由输出的结果我们可以发现第一个空白分隔的字段代表帐号,所以使用如上指令:
# 但是因为 root pts/1 之间空格有好几个,并非仅有一个,所以,如果要找出
# pts/1 其实不能以 cut -d ' ' -f 1,2 喔!输出的结果会不是我们想要的。
```

grep: 分析一行消息,若该行有我们需要的信息,就拿出来


```
[dmtsai@study ~]$ grep [-acinv] [--color=auto] '搜寻字符串' filename
选项与参数：
-a : 将 binary 文件以 text 文件的方式搜寻数据
-c : 计算找到 '搜寻字符串' 的次数
-i : 忽略大小写的不同，所以大小写视为相同
-n : 顺便输出行号
-v : 反向选择，亦即显示出没有 '搜寻字符串' 内容的那一行！
--color=auto : 可以将找到的关键字部分加上颜色的显示喔！
```

范例一：将 last 当中，有出现 root 的那一行就取出来；

```
[dmtsai@study ~]$ last &#124; grep 'root'
```

范例二：与范例一相反，只要没有 root 的就取出！

```
[dmtsai@study ~]$ last &#124; grep -v 'root'
```

范例三：在 last 的输出讯息中，只要有 root 就取出，并且仅取第一栏

```
[dmtsai@study ~]$ last &#124; grep 'root' &#124; cut -d ' ' -f1
```

在取出 root 之后，利用上个指令 cut 的处理，就能够仅取得第一栏啰！

范例四：取出 /etc/man_db.conf 内含 MANPATH 的那几行

```
[dmtsai@study ~]$ grep --color=auto 'MANPATH' /etc/man_db.conf
```

.... (前面省略)

```
MANPATH_MAP      /usr/games                /usr/share/man
```

```
MANPATH_MAP      /opt/bin                  /opt/man
```

```
MANPATH_MAP      /opt/sbin                 /opt/man
```

神奇的是，如果加上 --color=auto 的选项，找到的关键字部分会用特殊颜色显示喔！

| 是 | 管道符

```
[dmtsai@study ~]$ grep [-A] [-B] [--color=auto] '搜寻字符串' filename
```

选项与参数：

-A : 后面可加数字，为 after 的意思，除了列出该行外，后续的 n 行也列出来；

-B : 后面可加数字，为 befer 的意思，除了列出该行外，前面的 n 行也列出来；

--color=auto 可将正确的那个撷取数据列出颜色

范例一：用 dmesg 列出核心讯息，再以 grep 找出内含 qxl 那行

```
[dmtsai@study ~]$ dmesg &#124; grep 'qxl'
```

```
[ 0.522749] [drm] qxl: 16M of VRAM memory size
```

```
[ 0.522750] [drm] qxl: 63M of IO pages memory ready (VRAM domain)
```

```
[ 0.522750] [drm] qxl: 32M of Surface memory size
```

```
[ 0.650714] fbcon: qxldrmfb (fb0) is primary device
```

```
[ 0.668487] qxl 0000:00:02.0: fb0: qxldrmfb frame buffer device
```

dmesg 可列出核心产生的讯息！包括硬件侦测的流程也会显示出来。

鸟哥使用的显卡是 QXL 这个虚拟卡，通过 grep 来 qxl 的相关信息，可发现如上信息。

范例二：承上题，要将捉到的关键字显色，且加上行号来表示：

```
[dmtsai@study ~]$ dmesg &#124; grep -n --color=auto 'qxl'
```

```
515:[ 0.522749] [drm] qxl: 16M of VRAM memory size
```

```
516:[ 0.522750] [drm] qxl: 63M of IO pages memory ready (VRAM domain)
```

```
517:[ 0.522750] [drm] qxl: 32M of Surface memory size
```

```
529:[ 0.650714] fbcon: qxldrmfb (fb0) is primary device
```

```
539:[ 0.668487] qxl 0000:00:02.0: fb0: qxldrmfb frame buffer device
```

除了 qxl 会有特殊颜色来表示之外，最前面还有行号喔！其实颜色显示已经是默认在 alias 当中了！

范例三：承上题，在关键字所在行的前两行与后三行也一起捉出来显示

```
[dmtsai@study ~]$ dmesg &#124; grep -n -A3 -B2 --color=auto 'qxl'
```

你会发现关键字之前与之后的数行也被显示出来！这样可以让你将关键字前后数据捉出来进行分析啦！

特殊字符：

特殊符号	代表意义
[[:alnum:]]	代表英文大小写字符及数字，亦即 0-9, A-Z, a-z
[[:alpha:]]	代表任何英文大小写字符，亦即 A-Z, a-z
[[:blank:]]	代表空白键与 [Tab] 按键两者
[[:cntrl:]]	代表键盘上面的控制按键，亦即包括 CR, LF, Tab, Del.. 等等
[[:digit:]]	代表数字而已，亦即 0-9
[[:graph:]]	除了空白字符（空白键与 [Tab] 按键）外的其他所有按键
[[:lower:]]	代表小写字符，亦即 a-z
[[:print:]]	代表任何可以被打印出来的字符
[[:punct:]]	代表标点符号（punctuation symbol），亦即："'?!;:#\$%&'...
[[:upper:]]	代表大写字符，亦即 A-Z
[[:space:]]	任何会产生空白的字符，包括空白键, [Tab], CR 等等
[[:xdigit:]]	代表 16 进位的数字类型，因此包括：0-9, A-F, a-f 的数字与字符

尤其上表中的[[:alnum:]], [[:alpha:]], [[:upper:]], [[:lower:]], [[:digit:]] 这几个一定要知道代表什么意思，因为他要比 a-z 或 A-Z 的用途要确定的很！好了，下面就让我们开始来玩玩进阶版的 grep 吧！

基础正则表达式字符汇整

11.2.4 基础正则表达式中的特殊字符 (Characters)

经过了上面的几个简单的范例，我们可以将基础的正则表达式特殊字符汇整如下：

RE 字符	意义与范例
<code>^word</code>	意义：待搜寻的字串 (word) 在行首！范例：搜寻行首为 # 开始的那一行，并列出行号 <code>> grep -n '^#' regular_express.txt</code>
<code>word\$</code>	意义：待搜寻的字串 (word) 在行尾！范例：将行尾为 ! 的那一行打印出来，并列出行号 <code>> grep -n '!\$' regular_express.txt</code>
<code>.</code>	意义：代表“一定有一个任意字符”的字符！范例：搜寻的字串可以是 (eve) (eae) (eee) (e e)，但不能仅有 (ee)！亦即 e 与 e 中间“一定”仅有一个字符，而空白字符也是字符！ <code>> grep -n 'e.e' regular_express.txt</code>
<code>\</code>	意义：跳脱字符，将特殊符号的特殊意义去除！范例：搜寻含有单引号 ' 的那一行！ <code>> grep -n \"'\" regular_express.txt</code>
<code>*</code>	意义：重复零个到无穷多个的前一个 RE 字符 范例：找出含有 (es) (ess) (esss) 等等的字串，注意，因为 可以是 0 个，所以 es 也是符合带搜寻字串。另外，因为 为重复“前一个 RE 字符”的符号，因此，在 之前必须要紧接着一个 RE 字符喔！例如任意字符则为 “.”！ <code>> grep -n 'ess*' regular_express.txt</code>
<code>[list]</code>	意义：字符集合的 RE 字符，里面列出想要撷取的字符！范例：搜寻含有 (gl) 或 (gd) 的那一行，需要特别留意的是，在 [] 当中“谨代表一个待搜寻的字符”，例如“a[afly]”代表搜寻的字串可以是 aay, afy, aly 即 [afly] 代表 a 或 f 或 l 的意思！ <code>> grep -n 'g[ld]' regular_express.txt</code>
<code>[n1-n2]</code>	意义：字符集合的 RE 字符，里面列出想要撷取的字符范围！范例：搜寻含有任意数字的那一行！需特别留意，在字符集合 [] 中的减号 - 是有特殊意义的，他代表两个字符之间的所有连续字符！但这个连续与否与 ASCII 编码有关，因此，你的编码需要设置正确（在 bash 当中，需要确定 LANG 与 LANGUAGE 的变量是否正确！）例如所有大写字符则为 [A-Z] <code>> grep -n '[A-Z]' regular_express.txt</code>
<code>[^list]</code>	意义：字符集合的 RE 字符，里面列出不要的字串或范围！范例：搜寻的字串可以是 (oog) (ood) 但不能是 (oot)，那个 ^ 在 [] 内时，代表的意义是“反向选择”的意思。例如，我不要大写字符，则为 [^A-Z]。但是，需要特别注意的是，如果以 <code>grep -n [^A-Z] regular_express.txt</code> 来搜寻，却发现该文件内的所有行都被列出，为什么？因为这个 [^A-Z] 是“非大写字符”的意思，因为每一行均有非大写字符，例如第一行的 "Open Source" 就有 p,e,n,o.... 等等的小写字 <code>> grep -n 'oo[^t]' regular_express.txt</code>
<code>{n,m}</code>	意义：连续 n 到 m 个的“前一个 RE 字符” 意义：若为 {n} 则是连续 n 个的前一个 RE 字符，意义：若是 {n,} 则是连续 n 个以上的前一个 RE 字符！范例：在 g 与 g 之间有 2 个到 3 个的 o 存在的字串，亦即 (goog) (gooog) <code>> grep -n 'go{2,3}g' regular_express.txt</code>

sed工具：

```
[dmtsai@study ~]$ sed [-nefr] [动作]
```

选项与参数：

- n : 使用安静 (silent) 模式。在一般 sed 的用法中，所有来自 STDIN 的数据一般都会被列出到屏幕上。但如果加上 -n 参数后，则只有经过 sed 特殊处理的那一行 (或者动作) 才会被列出来。
- e : 直接在命令行界面上进行 sed 的动作编辑；
- f : 直接将 sed 的动作写在一个文件内，-f filename 则可以执行 filename 内的 sed 动作；
- r : sed 的动作支持的是延伸型正则表达式的语法。(默认是基础正则表达式语法)
- i : 直接修改读取的文件内容，而不是由屏幕输出。

动作说明： [n1[,n2]]function

n1, n2 : 不见得会存在，一般代表“选择进行动作的行数”，举例来说，如果我的动作是需要在 10 到 20 行之间进行的，则“ 10,20[动作行为] ”

function 有下面这些咚咚：

- a : 新增，a 的后面可以接字符串，而这些字符串会在新的一行出现 (目前的下一行) ~
- c : 取代，c 的后面可以接字符串，这些字符串可以取代 n1,n2 之间的行！
- d : 删除，因为是删除啊，所以 d 后面通常不接任何咚咚；
- i : 插入，i 的后面可以接字符串，而这些字符串会在新的一行出现 (目前的上一行) ；
- p : 打印，亦即将某个选择的数据印出。通常 p 会与参数 sed -n 一起运行~
- s : 取代，可以直接进行取代的工作哩！通常这个 s 的动作可以搭配正则表达式！
例如 1,20s/old/new/g 就是啦！

格式化打印：printf

```
[dmtsai@study ~]$ printf '打印格式' 实际内容
```

选项与参数：

关于格式方面的几个特殊样式：

- \a 警告声音输出
- \b 倒退键 (backspace)
- \f 清除屏幕 (form feed)
- \n 输出新的一行
- \r 亦即 Enter 按键
- \t 水平的 [tab] 按键
- \v 垂直的 [tab] 按键
- \xNN NN 为两位数的数字，可以转换数字成为字符。

关于 C 程序语言内，常见的变量格式

- %ns 那个 n 是数字，s 代表 string，亦即多少个字符；
- %ni 那个 n 是数字，i 代表 integer，亦即多少整数码数；
- %N.nf 那个 n 与 N 都是数字，f 代表 floating (浮点)，如果有小数码数，假设我共要十个位数，但小数点有两位，即为 %10.2f 啰！

另外，在 printf 后续的那一段格式中，%s 代表一个不固定长度的字串，而字串与字串中间就以 \t 这个 [tab] 分隔符号来处理！你要记得的是，由于 \t 与 %s 中间还有空格，因此每个字串间会有一个 [tab] 与一个空白键的分隔喔！

既然每个字段的长度不固定会造成上述的困扰，那我将每个字段固定就好啦！没错没错！这样想非常好！所以我们就将数据给他进行固定字段长度的设计吧！

范例二：将上述数据关于第二行以后，分别以字串、整数、小数点来显示：

```
[dmtsai@study ~]$ printf '%10s %5i %5i %5i %8.2f \n' $(cat printf.txt &#124; grep -v Nam
DmTsay      80      60      92      77.33
VBird       75      55      80      70.00
Ken         60      90      70      73.33
```

上面这一串格式想必您看得很辛苦！没关系！一个一个来解释！上面的格式共分为五个字段，%10s 代表的是一个长度为 10 个字符的字串字段，%5i 代表的是长度为 5 个字符的数字字段，至于那个 %8.2f 则代表长度为 8 个字符的具有小数点的字段，其中小数点有两个字符宽度。我们可以使用下面的说明来介绍 %8.2f 的意义：

字符宽度：12345678 %8.2f 意义：00000.00

如上所述，全部的宽度仅有 8 个字符，整数部分占有 5 个字符，小数点本身 (.) 占一位，小数点下的位数则有两位。这种格式经常使用于数值程序的设计中！这样了解乎？自己试看看如果要将小数点位数变成 1 位又该如何处理？

awk:数据处理工具

awk 也是一个非常棒的数据处理工具！相较于 sed 常常作用于一整个行的处理，awk 则比较倾向于一行当中分成数个“字段”来处理。因此，awk 相当的适合处理小型的数据数据处理呢！awk 通常运行的模式是这样的：

```
[dmtsai@study ~]$ awk '条件类型1{动作1} 条件类型2{动作2} ...' filename
```

awk 后面接两个单引号并加上大括号 {} 来设置想要对数据进行的处理动作。awk 可以处理后续接的文件，也可以读取来自前个指令的 standard output。但如前面说的，awk 主要是处理“每一行的字段内的数据”，而默认的“字段的分隔符号为“空白键”或 “[tab]键””！举例来说，我们用 last 可以将登陆者的数据取出来，结果如下所示：

```
[dmtsai@study ~]$ last -n 5 &lt;==仅取出前五行
dmtsai pts/0      192.168.1.100  Tue Jul 14 17:32  still logged in
dmtsai pts/0      192.168.1.100  Thu Jul  9 23:36 - 02:58  (03:22)
dmtsai pts/0      192.168.1.100  Thu Jul  9 17:23 - 23:36  (06:12)
dmtsai pts/0      192.168.1.100  Thu Jul  9 08:02 - 08:17  (00:14)
dmtsai tty1        Fri May 29 11:55 - 12:11  (00:15)
```

若我想要取出帐号与登陆者的 IP，且帐号与 IP 之间以 [tab] 隔开，则会变成这样：

```
[dmtsai@study ~]$ last -n 5 &#124; awk '{print $1 "\t" $3}'
dmtsai 192.168.1.100
dmtsai 192.168.1.100
dmtsai 192.168.1.100
dmtsai 192.168.1.100
dmtsai Fri
```

上表是 awk 最常使用的动作！通过 print 的功能将字段数据列出来！字段的分隔则以空白键或 [tab] 按键来隔开。因为不论哪一行我都要处理，因此，就不需要有“条件类型”的限制！我想要的是第一栏以及第三栏，但是，第五行的内容怪怪的～这是因为数据格式的问题啊！所

- 正则表达式就是处理字串的方法，他是以行为单位来进行字串的处理行为；
- 正则表达式通过一些特殊符号的辅助，可以让使用者轻易的达到“搜寻/删除/取代”某特定字串的处理程序；
- 只要工具程序支持正则表达式，那么该工具程序就可以用来作为正则表达式的字串处理之用；
- 正则表达式与万用字符是完全不一样的东西！万用字符（wildcard）代表的是 bash 操作接口的一个功能，但正则表达式则是一种字串处理的表示方式！
- 使用 grep 或其他工具进行正则表达式的字串比对时，因为编码的问题会有不同的状态，因此，你最好将 LANG 等变量设置为 C 或者是 en 等英文语系！
- grep 与 egrep 在正则表达式里面是很常见的两支程序，其中，egrep 支持更严谨的正则表达式的语法；
- 由于编码系统的不同，不同的语系（LANG）会造成正则表达式撷取数据的差异。因此可利用特殊符号如 [:upper:] 来替代编码范围较佳；
- 由于严谨度的不同，正则表达式之上还有更严谨的延伸正则表达式；
- 基础正则表达式的特殊字符有：*, ., [], [-], ^, \$ 等！
- 常见的支持正则表达式的工具软件有：grep, sed, vim 等等
- printf 可以通过一些特殊符号来将数据进行格式化输出；
- awk 可以使用“字段”为依据，进行数据的重新整理与输出；
- 文件的比对中，可利用 diff 及 cmp 进行比对，其中 diff 主要用在纯文本方面的新旧版本比对
- patch 指令可以将旧版数据更新到新版（主要亦由 diff 创建 patch 的补丁来源文件）